

B1: Why Permissioned Data

AT Protocol (atproto) gets called a "protocol," but it's really a framework: several distinct protocols plus a schema language. What atproto ships today is a data repository and sync protocol for **public broadcast** data. Users publish records into a repository on their PDS, applications crawl those repositories to build views, and every record is signed, redistributable, and universally addressable, with authority resting in the DID that publishes it.

The Permissioned Data proposal (number 0016) specifies a second data protocol alongside public broadcast, for data that must not be world-readable. It's a proposal, not a final specification. Details, terminology, and behaviors are all likely to change.

This document specifies an additional data protocol for **permissioned data**, or data with an access perimeter.

Permissioned Data proposal, "Introduction"

The data modalities

"Permissioned data" covers more than one use case. The proposal lists the modalities the protocol is built to serve, from strictly single-user data up to communities of many users:

- **Personal data:** bookmarks, muted, drafts
- **Gated content:** paid newsletters, subscriber-only posts
- **Socially shared:** private posts, stories
- **Groups:** private forums, communities, group chats

Ordinary public posts are not on this list. Data meant for open, world-readable distribution stays on the existing public broadcast protocol. Permissioned data exists for the cases where an access perimeter is required, whether that perimeter is "only me" (personal data), "paying subscribers" (gated content), "people I chose" (socially shared), or "members of this community" (groups). One protocol design covers all four. The proposal distinguishes them mainly by how the space is configured and who its authority is, not by separate mechanisms per modality.

Access control vs confidentiality

The proposal states its central promise, and its central non-promise, up front:

?

The permissioned data protocol provides **access control, not confidentiality**. It is not end-to-end encrypted. Services (both PDSes and authorized applications) can read the data they handle, which is required for server-side features such as search, indexing, notifications, aggregation, and moderation. E2EE is a separate concern that may be layered on top by an application and is out of scope in this proposal.

Permissioned Data proposal, "Introduction"

The distinction does a lot of work. Access control means the protocol decides who can get the data: reading a space takes a credential, and the space's authority decides whether you get one. Confidentiality would mean no server can read the content, and the protocol makes no such promise. Records sit in plaintext on the services that handle them. Your PDS can read your bookmarks. An app admitted to a private forum can read the forum.

That's a deliberate trade, not an oversight. Search, indexing, notifications, aggregation, and moderation all need a service to inspect record content. Hide the content from every server and none of those features can be built server-side. If your threat model requires that even authorized infrastructure can't read the content, that's an end-to-end encryption problem, and the proposal leaves it to applications: E2EE may be layered on top, but it's out of scope. So treat permissioned data as a gatekeeping layer, not a secrecy layer.

Relationship to public broadcast

The permissioned data protocol is deliberately shaped like the protocol you already know:

The permissioned data protocol shares the abstract shape of public broadcast. It retains DID-based authority, per-user repositories, lexicon-typed records, and the general flow of applications crawling PDSes to build views. However it has its own repository format, sync mechanism, addressing scheme, and resolution path. Public broadcast is built for open distribution (signed, archival, rebroadcastable) while the permissioned data protocol is built for party-to-party transmission within an access boundary.

Permissioned Data proposal, "Introduction"

So the mental model carries over: DIDs, per-user repos, Lexicon records, applications crawling PDSes. The concrete machinery is replaced everywhere the open-distribution assumption baked into public broadcast would leak data or make it permanently rebroadcastable. The proposal summarizes the differences in one table:

	Public broadcast	Permissioned data
Unit of data	Record in a repo	Record in a permissioned repo
Repo scope	One repo per user	One permissioned repo per (user, space)
Record authority	User DID	User DID

	Public broadcast	Permissioned data
URI authority	User DID	Space authority DID
Commit	Merkle Search Tree root	LtHash set-hash digest
Signature	Rebroadcastable, archival	Deniable on rebroadcast
Addressing	Traditional at:// URI	at:// URI with space segment
Access	Public	Gated by space credential

Permissioned Data proposal, "Relationship to public broadcast"

Repo scope: per user vs per (user, space)

In public broadcast a user has one repo. In permissioned data a user has one permissioned repo for each space they participate in. The unit of storage is the (user, space) pair: each permissioned repo holds one user's records within one space, hosted on that user's own PDS. A user in ten private forums has ten separate permissioned repos. A space never colocates its records on a single host; it is the aggregation of every member's per-user repo.

Record authority vs URI authority

These are separate rows for a reason. Record authority, meaning whose record it is, who authored and controls it, stays with the user DID in both protocols. URI authority, the DID that anchors the record's address, changes: in permissioned data it is the *space authority DID*, not the author's DID. A permissioned record's URI starts from the space it lives in, then names the author within it. The author still owns the record; the space anchors its address.

Commit: MST root vs LtHash digest

Public broadcast repos commit to a Merkle Search Tree root. Permissioned repos have their own repository format and commit to an **LtHash set-hash digest** instead. The mechanics of LtHash are covered in section B4; for now it's enough to know which protocol uses which structure.

Signature: archival vs deniable

A public broadcast record's signature is rebroadcastable and archival. Anyone can pass it along and it stays verifiable forever, which is exactly what open distribution wants. A permissioned record's signature is **deniable on rebroadcast**. The protocol is built for party-to-party transmission within an access boundary, so if data leaks outside that boundary, the signature gives third parties no durable cryptographic proof of authorship. How deniable commit signatures work is section B4 material.

Addressing and access

Public records use the traditional **at://** URI. Permissioned records reuse the **at://** scheme but insert a fixed **space** segment (section B2 covers the grammar). And where public data is readable by anyone, reading permissioned data is gated by a **space credential** (section B3).

Terminology

The proposal defines its vocabulary in one block. Every later section leans on these terms, so get them exact:

- **Space:** an authorization and sync boundary for a set of permissioned records, identified by an (**authority**, **type**, **skey**) triple.
- **Permissioned repo:** one user's records within one space, with a cryptographic commit, hosted on that user's PDS.
- **Repo host:** a service that stores and serves users' permissioned repos.
- **Space host:** a service that answers for a space as a whole, issuing credentials, enumerating writers, and routing notifications.
- **Space authority:** the DID at the root of a space, which resolves to the space host and the key material for issuing credentials.
- **Space credential:** a token issued by the space authority that grants read access to a space.
- **Delegation token:** a token issued by a user's PDS that an application exchanges with a space authority for a space credential.
- **Client attestation:** a token signed by an application's own client authentication key, proving the application's identity to a space authority. Required only when a space gates on client app identity.
- **Syncer:** an application that keeps its own copy of a space in sync by pulling from repo hosts.

Permissioned Data proposal, "Terminology"

Telling the roles apart

A repo host and a space host are two different service roles. A repo host stores and serves individual users' permissioned repos; it deals in per-user data. A space host answers for the space as a whole: it issues credentials, enumerates the space's writers, and routes notifications. When you need "who is in this space" or "give me a credential," you talk to the space host. When you need "give me this member's records," you talk to that member's repo host.

The space authority is a DID, not a service. It is the identity at the root of the space. Resolving that DID yields the space host endpoint and the key material used to verify the space's credentials. The authority is who the space answers to; the host is the service doing the answering.

Three tokens appear in the terminology, and they slot together. An application gets a *delegation token* from the user's PDS and exchanges it with the space authority for a *space credential*. When a space gates on client app identity, the application also presents a *client attestation* signed by its own client authentication key. The space credential is what actually grants read access. Section B3 walks the full flow.

A syncer is a consumer role: an application that maintains its own copy of a space by pulling each member's permissioned repo from the repo hosts. It reads the space; it does not answer for it.

Does a space have to live on a PDS?

No, and the proposal names the roles separately for exactly that reason:

A PDS fulfills both the roles of a **repo host** and a **space host**. However, these roles are discussed separately because they do not necessarily need to be filled by a PDS. A permissioned repo or a space may be hosted by any service that implements the required APIs.

Permissioned Data proposal, "Terminology"

In the common case your PDS is both your repo host and a space host. But the roles are defined by API surface, not by being a PDS. Any service that implements the required APIs can host a permissioned repo or a space.

You should now be able to explain

- What "permissioned data" means (data with an access perimeter) and the four modalities the protocol serves: personal data, gated content, socially shared data, and groups
- Why public posts are not a permissioned-data modality: they stay on the existing public broadcast protocol
- That the protocol provides access control, not confidentiality, and is not end-to-end encrypted
- Which services can read permissioned data (PDSes and authorized applications) and why that is required (search, indexing, notifications, aggregation, moderation)
- That E2EE is a separate concern an application may layer on top, out of scope in the proposal
- What permissioned data keeps from public broadcast (DID-based authority, per-user repositories, lexicon-typed records, applications crawling PDSes) and what it replaces (repository format, sync mechanism, addressing scheme, resolution path)
- Repo scope: one repo per user in public broadcast, one permissioned repo per (user, space) in permissioned data
- Record authority vs URI authority: the user DID authors the record in both protocols, but a permissioned record's URI is anchored on the space authority DID
- Commits: a Merkle Search Tree root in public broadcast, an LtHash set-hash digest in permissioned data
- Signatures: rebroadcastable and archival in public broadcast, deniable on rebroadcast in permissioned data, matching open distribution vs party-to-party transmission within an access boundary
- The core vocabulary: space, permissioned repo, repo host, space host, space authority, space credential, delegation token, client attestation, syncer
- How the space host (a service answering for the space as a whole) differs from the repo host (a service serving individual users' repos), and how both differ from the space authority (a DID)

- That a PDS fills both hosting roles by default, but any service implementing the required APIs may host a permissioned repo or a space